

Relatório

Exercício 1 – Menu: opção 1:

Interface de desenvolvimento para o processador MARS 4.5. O arquivo de código fonte é `/home/jem/Documentos/AOC/pratica2/menu.asm`.

Text Segment:

Bkpt	Address	Code	Basic	Source
0x00400000	0x3c011001	lui \$1, 0x00001001	18: la \$a0, menuText	
0x00400004	0x34240000	ori \$4, 0x00000000	19: addi \$v0, \$zero, 4	
0x00400008	0x20020004	addi \$2, \$0, 0x00000004	20: syscall	
0x0040000c	0x00000000	syscall	23: la \$a0, opcaoText	
0x00400010	0x3c011001	lui \$1, 0x00001001		
0x00400014	0x34240000	ori \$4, 0x00000000	24: addi \$v0, \$zero, 4	
0x00400018	0x20020004	addi \$2, \$0, 0x00000004	25: syscall	
0x0040001c	0x00000000	syscall	27: addi \$v0, \$zero, 5	
0x00400020	0x20020005	addi \$2, \$0, 0x00000005	28: syscall	
0x00400024	0x00000000	syscall	29: add \$s0, \$zero, \$v0	
0x00400028	0x00020002	add \$16, \$0, \$2	30: slti \$t0, \$s0, 1	
0x0040002c	0x2a080001	slti \$8, \$16, 0x00000001	31: bne \$t0, \$zero, valorErrado	
0x00400030	0x15000004	bne \$s, \$0, 0x00000004	32: addi \$t1, \$zero, 3	
0x00400034	0x20090003	addi \$9, \$0, 0x00000003		

Data Segment:

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x6f637345	0x2061689c	0x20616d75	0x20736164	0xf5e7706f	0x61207365	0x67657320	0x0a726975
0x10010020	0x202d2031	0x69697270	0x6f726965	0x6d6a6e20	0x206f7265	0x69676964	0x6f646174	0x6d20e520
0x10010040	0x726f6961	0x206f6a20	0x20657571	0x75732061	0x61727462	0x206f6a37	0x6d206f64	0x726f6961
0x10010060	0x6f6f6570	0x206f7265	0x6f6c6570	0x6e656d20	0x6a37726f	0x202d2032	0x616d6f53	0x69727474
0x10010080	0x20320a6f	0x6553202d	0x6e657571	0x00616963	0x6769440a	0x20657469	0x706f7261	0x3a6f6a37
0x100100a0	0x4e4a0020	0x72656f6a	0x6e65206f	0x696c6178	0x6d206f64	0x74696769	0x756e2065	0x6f726569
0x100100c0	0x65642073	0x6120120	0x4a003320	0x74696769	0x206f7265	0x6d697270	0x6f726965	0x6d6f6a20
0x100100e0	0x3a6f7265	0x6d440020	0x65746967	0x73206f20	0x6e756765	0x6e206f64	0x72656d75	0x00203a6f
0x10010100	0x69697644	0x75206574	0x74692066	0x6f72656d	0x6f70003a	0x73657220	0x61746e75	0x69696f64
0x10010120	0x2002003a	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010140	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010160	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010180	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100101a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100101c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100101e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

Mars Messages:

Escolha uma das opções a seguir
1 - primeiro número digitado é maior do que a subtração do maior número pelo menor?
2 - Sequência
3 - Sequência
Digite a opção: 1
Digite o primeiro número: 4
Digite o segundo número: 6
-- program is finished running --

opção 2:

Interface de desenvolvimento para o processador MARS 4.5. O arquivo de código fonte é `/home/jem/Documentos/AOC/pratica2/menu.asm`.

Text Segment:

Bkpt	Address	Code	Basic	Source
0x00400000	0x3c011001	lui \$1, 0x00001001	18: la \$a0, menuText	
0x00400004	0x34240000	ori \$4, 0x00000000	19: addi \$v0, \$zero, 4	
0x00400008	0x20020004	addi \$2, \$0, 0x00000004	20: syscall	
0x0040000c	0x00000000	syscall	23: la \$a0, opcaoText	
0x00400010	0x3c011001	lui \$1, 0x00001001		
0x00400014	0x34240000	ori \$4, 0x00000000	24: addi \$v0, \$zero, 4	
0x00400018	0x20020004	addi \$2, \$0, 0x00000004	25: syscall	
0x0040001c	0x00000000	syscall	27: addi \$v0, \$zero, 5	
0x00400020	0x20020005	addi \$2, \$0, 0x00000005	28: syscall	
0x00400024	0x00000000	syscall	29: add \$s0, \$zero, \$v0	
0x00400028	0x00020002	add \$16, \$0, \$2	30: slti \$t0, \$s0, 1	
0x0040002c	0x2a080001	slti \$8, \$16, 0x00000001	31: bne \$t0, \$zero, valorErrado	
0x00400030	0x15000004	bne \$s, \$0, 0x00000004	32: addi \$t1, \$zero, 3	
0x00400034	0x20090003	addi \$9, \$0, 0x00000003		

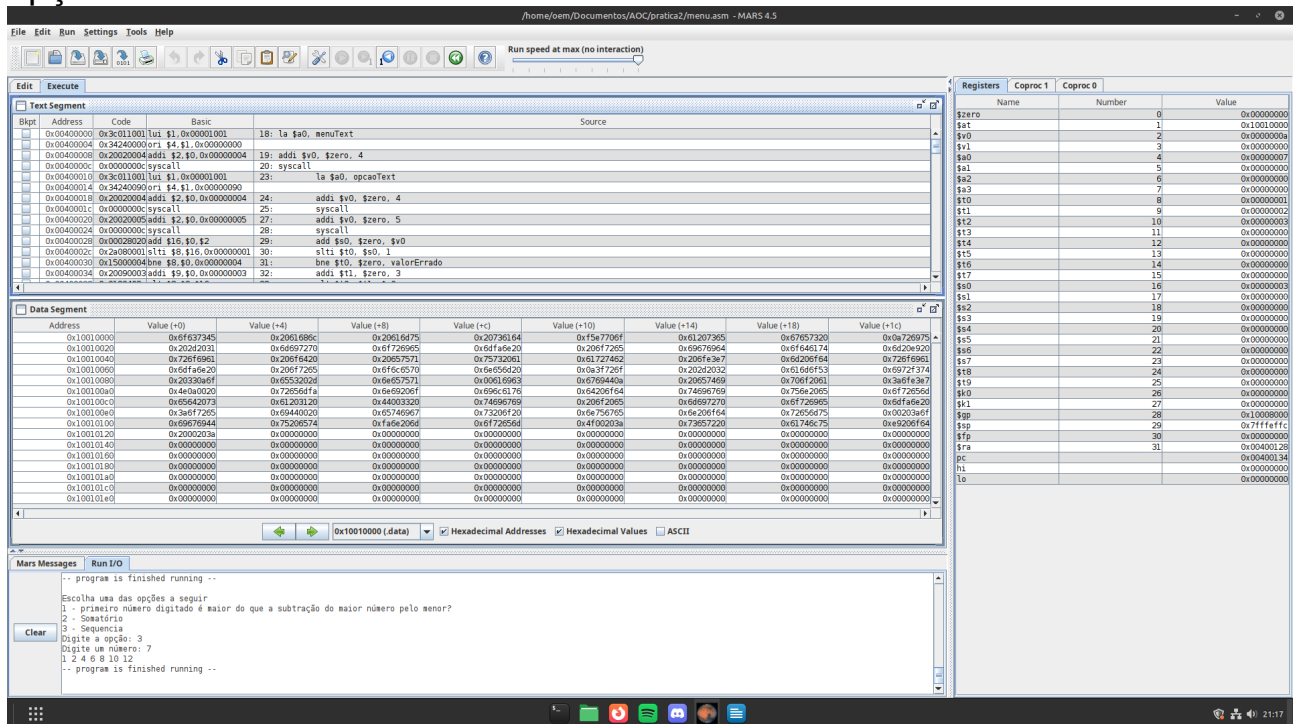
Data Segment:

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x6f637345	0x2061689c	0x20616d75	0x20736164	0xf5e7706f	0x61207365	0x67657320	0x0a726975
0x10010020	0x202d2031	0x69697270	0x6f726965	0x6d6a6e20	0x206f7265	0x69676964	0x6f646174	0x6d20e520
0x10010040	0x726f6961	0x206f6a20	0x20657571	0x75732061	0x61727462	0x206f6a37	0x6d206f64	0x726f6961
0x10010060	0x6f6f6570	0x206f7265	0x6f6c6570	0x6e656d20	0x6a37726f	0x202d2032	0x616d6f53	0x69727474
0x10010080	0x20320a6f	0x6553202d	0x6e657571	0x00616963	0x6769440a	0x20657469	0x706f7261	0x3a6f6a37
0x100100a0	0x4e4a0020	0x72656f6a	0x6e65206f	0x696c6178	0x6d206f64	0x74696769	0x756e2065	0x6f726569
0x100100c0	0x65642073	0x6120120	0x4a003320	0x74696769	0x206f7265	0x6d697270	0x6f726965	0x6d6f6a20
0x100100e0	0x3a6f7265	0x6d440020	0x65746967	0x73206f20	0x6e756765	0x6e206f64	0x72656d75	0x00203a6f
0x10010100	0x69697644	0x75206574	0x74692066	0x6f72656d	0x6f70003a	0x73657220	0x61746e75	0x69696f64
0x10010120	0x2002003a	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010140	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010160	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010180	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100101a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100101c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100101e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

Mars Messages:

-- program is finished running --
Escolha uma das opções a seguir
1 - primeiro número digitado é maior do que a subtração do maior número pelo menor?
2 - Sequência
3 - Sequência
Digite a opção: 23
O resultado é: 276
-- program is finished running --

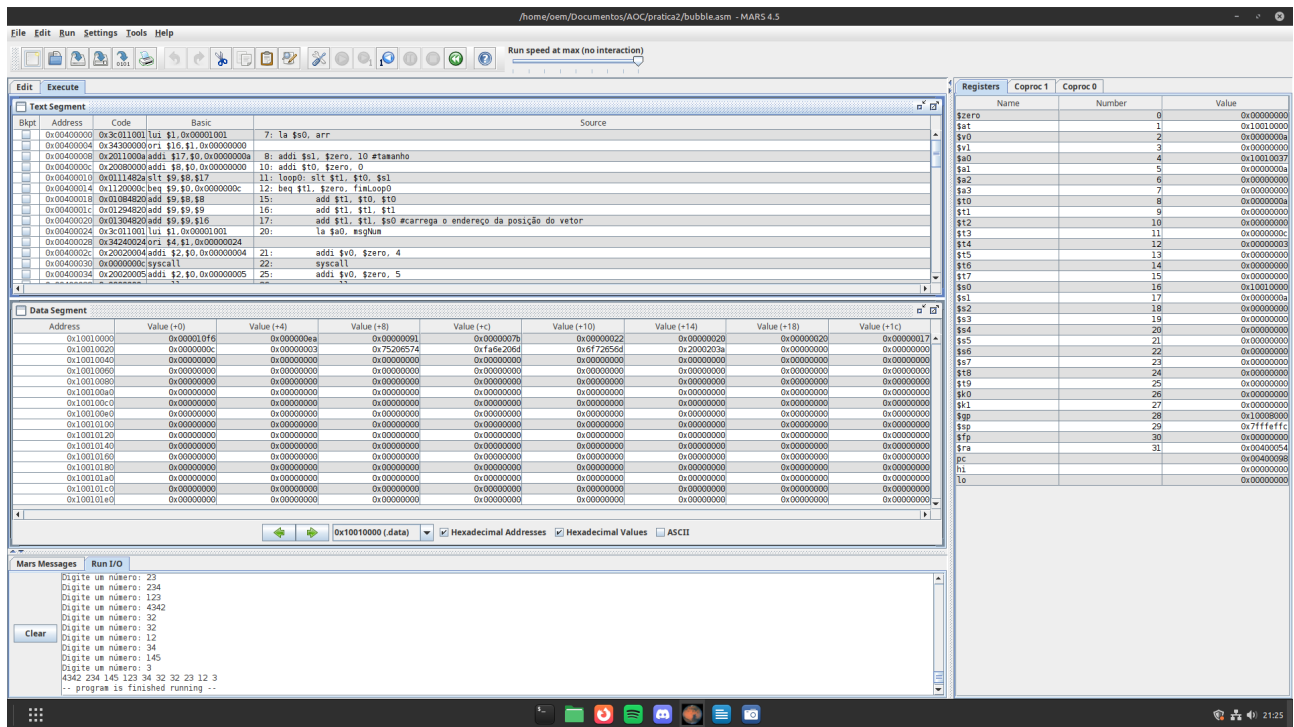
opção 3:



O código do Exercício 1 foi estruturado utilizando um laço de repetição (etiqueta `doWhile`) para garantir a validação da entrada do usuário, testando os limites e impedindo que opções menores que 1 ou maiores que 3 sejam aceitas, conforme exigido. A decisão principal para o controle do fluxo de execução foi a implementação de um desvio condicional do tipo `switch-case` com instruções `beq`, que direciona o programa para as três opções distintas. Além disso, a rotina sequencia demonstra boas práticas no uso da memória, salvando e restaurando parâmetros e registradores locais na pilha (`$sp`) para preservar os dados originais durante a execução da função.

As imagens de execução do Exercício 1 evidenciam que o menu interage corretamente com o usuário, validando os inputs e desviando para a rotina correta. Através das capturas de tela, fica comprovado o processamento adequado da lógica de cada bloco, com destaque para a opção 2, que computa corretamente o somatório do número 23 (resultando em 276), e para a opção 3, que imprime perfeitamente os 7 primeiros termos da sequência requerida (1, 2, 4, 6, 8, 10, 12).

Exercício 2 – bubble sort:



No Exercício 2, a principal decisão de projeto foi modularizar o algoritmo de ordenação dentro de uma sub-rotina independente chamada `bSort`, que recebe o endereço base do vetor em `$a0` e o seu tamanho em `$a1`. Para atender ao requisito do roteiro de imprimir os dados em ordem decrescente, a lógica de comparação interna utiliza a instrução `slt` para testar se o valor na posição atual é menor que o valor na posição seguinte. Se for, o programa realiza a troca dos valores na memória (swap) e altera uma flag (registrador `$s0`) para sinalizar que o vetor ainda não está totalmente ordenado.

A captura de tela do Bubble Sort evidencia a coleta bem-sucedida dos 10 números arbitrários informados pelo usuário. A imagem da execução comprova de forma direta que, após a chamada da sub-rotina, o vetor original é impresso no console estritamente em ordem decrescente (iniciando no maior valor, 4342, e finalizando no menor valor digitado, 3), validando o correto funcionamento das iterações dos laços do algoritmo

Exercício 3 – calculadora: soma:

Interface de desenvolvimento para o Mars (MARS 4.5) mostrando o código de uma calculadora em assembly MIPS. O código realiza operações de soma, subtração, multiplicação e exponenciação com base nas opções selecionadas pelo usuário.

Text Segment:

```
0x00400000: 0x0c180021 jal 0x00400084      16: jal lerDigito
0x00400004: 0x00028020 add $16,$0,$2      17: add $v0,$zero,$v0
0x00400008: 0x0c180021 jal 0x00400084      18: jal lerDigito
0x0040000c: 0x00028020 add $17,$0,$2      19: add $s1,$zero,$v0
0x00400010: 0x3c011001 lui $1,0x00001001 21: la $a0,$end1
0x00400014: 0x34240070 ori $4,$1,0x00000070
0x00400018: 0x00020004 addi $2,$0,0x00000004 22: addi $v0,$zero,4
0x0040001c: 0x0000000c syscall          23: syscall
0x00400020: 0x0c18002b jal 0x004000ac      26: jal lerOpcao
0x00400024: 0x00025020 add $5,$0,$2      29: add $a2,$zero,$v0
0x00400028: 0x00102020 add $4,$0,$16     30: add $a0,$zero,$a0
0x0040002c: 0x00112020 add $5,$0,$17     31: add $a1,$zero,$s1
0x00400030: 0x0c180085 jal 0x00400084      32: jal executarOp executar a operação
0x00400034: 0x00029020 add $18,$0,$2      34: add $s2,$zero,$v0
```

Data Segment:

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x69676944	0x75206574	0xfaf62064	0x6f726566	0x3100203a	0x6d6f7326	0x202d2061	0x75732d32
0x10010020	0x61727462	0x20f13e7	0x2d332026	0x746c756d	0x696c7069	0xe3e76163	0x202d206f	0x78652d34
0x10010040	0x69646f70	0x69646f70	0x69646f70	0x69646f70	0x69646f70	0x69646f70	0x69646f70	0x69646f70
0x10010060	0x3e76172	0x6969206f	0x696c176	0x00a6164	0x0000000a	0x00400130	0x00400138	0x00400140
0x10010080	0x00400170	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010100	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010120	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010140	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010160	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010180	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100101a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100101c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100101e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

Registers:

Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x10010000
\$v0	2	0x0000000a
\$v1	3	0x00000001
\$a0	4	0x00000008
\$a1	5	0x00000010
\$a2	6	0x00000002
\$a3	7	0x00000000
\$t0	8	0x00400130
\$t1	9	0x00000000
\$t2	10	0x00000000
\$t3	11	0x00000000
\$t4	12	0x00000000
\$t5	13	0x00000000
\$t6	14	0x00000000
\$t7	15	0x00000000
\$t8	16	0x00000018
\$t9	17	0x00000010
\$s0	18	0x00000008
\$s1	19	0x00000000
\$s2	20	0x00000000
\$s3	21	0x75732d32
\$s4	22	0x78652d34
\$s5	23	0x00000000
\$s6	24	0x00000000
\$s7	25	0x00000000
\$s8	26	0x00000000
\$s9	27	0x00000000
\$fp	28	0x10000000
\$gp	29	0x7ffffcfc
\$ra	30	0x00000000
\$pc	31	0x00400084
\$lo		0x00000000
\$hi		0x00000000

Mars Messages:

```
-- program is finished running --
Digite um número: 24
Digite um número: 16
1-soma - 2-subtração - 3-multiplicação - 4-exponencial
Digite uma opção: 1
40
-- program is finished running --
```

subtração:

Interface de desenvolvimento para o Mars (MARS 4.5) mostrando o código de uma calculadora em assembly MIPS. O código realiza operações de soma, subtração, multiplicação e exponenciação com base nas opções selecionadas pelo usuário.

Text Segment:

```
0x00400000: 0x0c180021 jal 0x00400084      16: jal lerDigito
0x00400004: 0x00028020 add $16,$0,$2      17: add $v0,$zero,$v0
0x00400008: 0x0c180021 jal 0x00400084      18: jal lerDigito
0x0040000c: 0x00028020 add $17,$0,$2      19: add $s1,$zero,$v0
0x00400010: 0x3c011001 lui $1,0x00001001 21: la $a0,$end1
0x00400014: 0x34240070 ori $4,$1,0x00000070
0x00400018: 0x00020004 addi $2,$0,0x00000004 22: addi $v0,$zero,4
0x0040001c: 0x0000000c syscall          23: syscall
0x00400020: 0x0c18002b jal 0x004000ac      26: jal lerOpcao
0x00400024: 0x00025020 add $5,$0,$2      29: add $a2,$zero,$v0
0x00400028: 0x00102020 add $4,$0,$16     30: add $a0,$zero,$a0
0x0040002c: 0x00112020 add $5,$0,$17     31: add $a1,$zero,$s1
0x00400030: 0x0c180085 jal 0x00400084      32: jal executarOp executar a operação
0x00400034: 0x00029020 add $18,$0,$2      34: add $s2,$zero,$v0
```

Data Segment:

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x69676944	0x75206574	0xfaf62064	0x6f726566	0x3100203a	0x6d6f7326	0x202d2061	0x75732d32
0x10010020	0x61727462	0x20f13e7	0x2d332026	0x746c756d	0x696c7069	0xe3e76163	0x202d206f	0x78652d34
0x10010040	0x69646f70	0x69646f70	0x69646f70	0x69646f70	0x69646f70	0x69646f70	0x69646f70	0x69646f70
0x10010060	0x3e76172	0x6969206f	0x696c176	0x00a6164	0x0000000a	0x00400130	0x00400138	0x00400140
0x10010080	0x00400170	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010100	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010120	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010140	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010160	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010180	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100101a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100101c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100101e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

Registers:

Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x10010000
\$v0	2	0x0000000a
\$v1	3	0x00000001
\$a0	4	0x00000008
\$a1	5	0x00000010
\$a2	6	0x00000002
\$a3	7	0x00000000
\$t0	8	0x00400130
\$t1	9	0x00000000
\$t2	10	0x00000000
\$t3	11	0x00000000
\$t4	12	0x00000000
\$t5	13	0x00000000
\$t6	14	0x00000000
\$t7	15	0x00000000
\$t8	16	0x00000018
\$t9	17	0x00000010
\$s0	18	0x00000008
\$s1	19	0x00000000
\$s2	20	0x00000000
\$s3	21	0x75732d32
\$s4	22	0x78652d34
\$s5	23	0x00000000
\$s6	24	0x00000000
\$s7	25	0x00000000
\$s8	26	0x00000000
\$s9	27	0x00000000
\$fp	28	0x10000000
\$gp	29	0x7ffffcfc
\$ra	30	0x00000000
\$pc	31	0x00400084
\$lo		0x00000000
\$hi		0x00000000

Mars Messages:

```
-- program is finished running --
Digite um número: 24
Digite um número: 16
1-soma - 2-subtração - 3-multiplicação - 4-exponencial
Digite uma opção: 2
8
-- program is finished running --
```

multiplicação:

The screenshot displays the MARS 4.5 calculator interface. The main window shows assembly code for a multiplication routine. The code includes instructions for reading digits, performing multiplication, and handling carry. The registers table on the right shows the state of various registers, including \$zero, \$at, \$v0, \$t1, \$a0, \$a1, \$a2, \$t0, \$t1, \$t2, \$t3, \$t4, \$t5, \$t6, \$t7, \$t8, \$t9, \$s0, \$s1, \$s2, \$s3, \$s4, \$s5, \$s6, \$s7, \$s8, \$s9, \$k0, \$k1, \$k2, \$k3, \$k4, \$k5, \$k6, \$k7, \$k8, \$k9, \$ra, \$sp, \$fp, \$ra, \$pc, \$hi, and \$lo. The console shows the program's output, including the input digits and the final result.

exponencial:

The screenshot displays the MARS 4.5 calculator interface. The main window shows assembly code for an exponential calculation routine. The code includes instructions for reading digits, performing exponential calculation, and handling carry. The registers table on the right shows the state of various registers, including \$zero, \$at, \$v0, \$t1, \$a0, \$a1, \$a2, \$t0, \$t1, \$t2, \$t3, \$t4, \$t5, \$t6, \$t7, \$t8, \$t9, \$s0, \$s1, \$s2, \$s3, \$s4, \$s5, \$s6, \$s7, \$s8, \$s9, \$k0, \$k1, \$k2, \$k3, \$k4, \$k5, \$k6, \$k7, \$k8, \$k9, \$ra, \$sp, \$fp, \$ra, \$pc, \$hi, and \$lo. The console shows the program's output, including the input digits and the final result.

O programa da calculadora foi desenvolvido com um alto nível de modularização, dividindo as tarefas específicas nas rotinas lerDigito, lerOpcao e de operações, obedecendo às restrições propostas pelo roteiro. Uma decisão técnica avançada neste código foi a estruturação de um array de ponteiros de função (operacoes), permitindo que a rotina executarOp calcule dinamicamente o endereço da operação desejada na memória e salte para ela utilizando a instrução jalr. Além disso, a operação de exponencial lida perfeitamente com o aninhamento de chamadas ao utilizar a pilha para

proteger o registrador de retorno (\$ra) e os parâmetros iniciais, permitindo que a sub-rotina chame a função de multiplicação (multi) consecutivas vezes dentro de um loop.

As imagens do relatório documentando a calculadora evidenciam o cumprimento funcional de todas as quatro operações matemáticas solicitadas. As execuções presentes nas capturas de tela atestam, de forma clara, que os laços baseados em somas sucessivas e multiplicações sucessivas funcionam muito bem, mesmo para operações que geram números elevados, o que é provado pelo acerto na rotina de exponencial onde 2 elevado a 24 gerou perfeitamente o resultado 16777216